

INVESTIGACION RESPECTOA A LOS METODOS DE LAS CLASES (SCANNER, STRING Y JAVA)

Java es un lenguaje de programación ampliamente utilizado en el desarrollo web, en la creación de aplicaciones y en sistemas empresariales debido a su robustez, seguridad y estabilidad. Una clase en Java es un modelo o plantilla que se utiliza para definir un tipo de objeto, indicando sus características mediante atributos y las acciones que puede realizar mediante métodos. Gracias a las clases, es posible organizar mejor el código, reutilizar funciones y representar elementos del mundo real dentro de un programa.

CLASE SCANNER:

Scanner en Java es una clase utilizada para facilitar la lectura de datos proporcionados por el usuario, como nombres, edades, números o textos. Es muy utilizada en programas que requieren interacción mediante la consola o el teclado.

La clase Scanner forma parte del paquete (java.util) y permite leer diferentes tipos de datos, como cadenas de texto, números enteros, números decimales y otros valores. Para utilizarla dentro de un programa es necesario importarla mediante (import java.util.Scanner;)

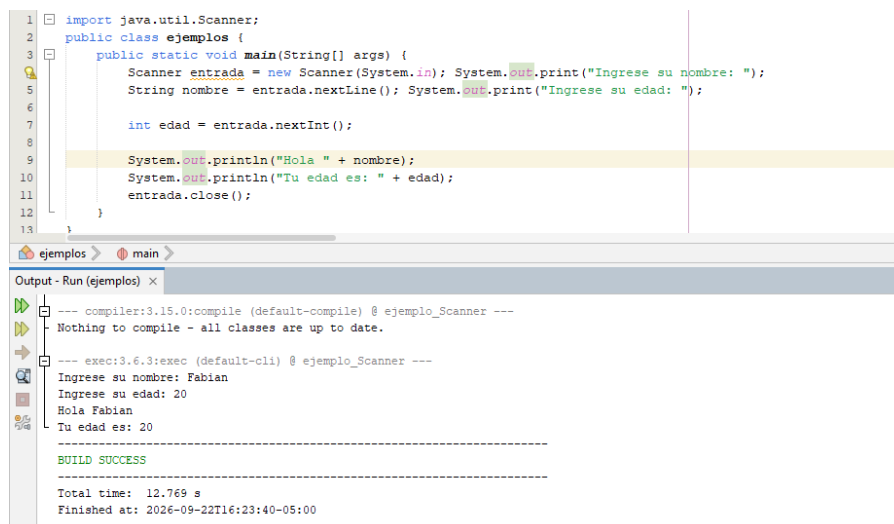
Scanner se encarga de leer e interpretar los datos provenientes de una fuente determinada, como el teclado, un archivo o una cadena de texto. Además, puede separar la información recibida en diferentes bloques para posteriormente procesarla mediante sus distintos métodos.

Esta clase fue incorporada en Java 5 con el propósito de facilitar la entrada y el procesamiento de datos, especialmente en programas que necesitan recibir valores ingresados por el usuario mediante la consola.

Al formar parte de Java, un lenguaje de programación orientado a objetos, Scanner se utiliza mediante la creación de objetos. Esto permite mantener una mejor organización del código y facilita el desarrollo de programas interactivos y fáciles de comprender.

A continuación, se presentan los principales métodos y se explica la función que cumple cada uno de ellos:

- **nextLine()** → Lee una línea completa de texto (direcciones, frases)
- **next()** → Lee una sola palabra (nombres, códigos)
- **nextInt()** → Lee un numero entero tipo int (edades, cantidades)
- **nextDouble()** → Lee un numero decimal tipo double (precios, notas)
- **hasNextInt()** → Verifica si el siguiente dato es número entero.
- **hasNext()** → Verifica si había más datos para leer.
- **close()** → Cierra el scanner cuando ya no se necesita.



```
1 import java.util.Scanner;
2 public class ejemplos {
3     public static void main(String[] args) {
4         Scanner entrada = new Scanner(System.in); System.out.print("Ingrese su nombre: ");
5         String nombre = entrada.nextLine(); System.out.print("Ingrese su edad: ");
6
7         int edad = entrada.nextInt();
8
9         System.out.println("Hola " + nombre);
10        System.out.println("Tu edad es: " + edad);
11        entrada.close();
12    }
13 }
```

ejemplos > main

Output - Run (ejemplos) x

```
--- compiler:3.15.0:compile (default-compile) @ ejemplo_Scanner ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ejemplo_Scanner ---
Ingrese su nombre: Fabian
Ingrese su edad: 20
Hola Fabian
Tu edad es: 20

BUILD SUCCESS

Total time: 12.769 s
Finished at: 2026-09-22T16:23:40:05:00
```

```

1 import java.util.Scanner;
2
3 public class ejemplos {
4
5     public static void main(String[] args) {
6
7         Scanner entrada = new Scanner(System.in);
8
9         System.out.print("Ingrese el nombre del producto: ");
10        String producto = entrada.nextLine();
11
12        System.out.print("Ingrese la cantidad: ");
13        int cantidad = entrada.nextInt();
14
15        System.out.print("Ingrese el precio unitario: ");
16        double precio = entrada.nextDouble();
17
18        double total = cantidad * precio;
19
20        System.out.println("Producto: " + producto);
21        System.out.println("Cantidad: " + cantidad);
22        System.out.println("Total a pagar: S/ " + total);
23
24        entrada.close();
25    }
26 }

```

➔

```

--- exec:3.6.3:exec (default-cli) @ ejemplo_Scanner ---
Ingrese el nombre del producto: Lavadora
Ingrese la cantidad: 2
Ingrese el precio unitario: 560
Producto: Lavadora
Cantidad: 2
Total a pagar: S/ 1120.0

BUILD SUCCESS

```

CLASE STRING:

La clase String en Java se utiliza para almacenar y trabajar con cadenas de texto, como nombres, palabras, mensajes o direcciones. Es una de las clases más utilizadas dentro del lenguaje Java debido a que permite manipular información textual de una manera sencilla.

Esta clase dispone de diferentes métodos que permiten examinar caracteres, comparar textos, buscar palabras, extraer partes de una cadena y convertir el contenido a mayúsculas o minúsculas. Por ejemplo, métodos como `length()`, `charAt()`, `equals()`, `contains()`, `substring()`, `toUpperCase()` y `toLowerCase()` facilitan el manejo de textos dentro de un programa.

Java también permite unir o concatenar cadenas utilizando el operador `+`. Por ejemplo:

- `String nombre = "Fabian";`
- `String saludo = "Hola " + nombre;`

Cuando se necesitan realizar muchas modificaciones o concatenaciones de texto, también se puede utilizar la clase `StringBuilder`, que permite trabajar con cadenas de forma más eficiente.

```

1 public class ejemplos {
2
3     public static void main(String[] args) {
4
5         String nombre = "Fabian";
6         String saludo = "Hola " + nombre;
7         System.out.println(saludo);
8     }
9 }

```

ejemplos

Output - Run (ejemplos) x

```

--- exec:3.6.3:exec (default-cli) @ ejemplo_Scanner ---
Hola Fabian

BUILD SUCCESS

Total time: 2.478 s
Finished at: 2026-09-23T15:17:04-05:00

```

Una característica importante de String es que sus objetos son inmutables, lo que significa que su contenido original no puede modificarse después de ser creado. Cuando se utiliza un método que aparentemente modifica el texto, Java realmente crea una nueva cadena con el resultado.

Por ejemplo:

- `String mensaje = "hola";`
- `String nuevoMensaje = mensaje.toUpperCase();`

En este caso, `mensaje` continúa teniendo el valor "hola", mientras que `nuevoMensaje` tendrá el valor "HOLA".

Por esta razón, la clase String es fundamental en Java, ya que proporciona diferentes métodos que permiten almacenar, analizar y manipular textos de manera sencilla y organizada.

```

1 public class ejemplos {
2
3     public static void main(String[] args) {
4
5         String mensaje = "hola";
6         String nm = mensaje.toUpperCase();
7
8         System.out.println("Texto original: " + mensaje);
9         System.out.println("Texto en mayúsculas: " + nm);
10    }
11 }

```

ejemplos

Output - Run (ejemplos) x

```

Texto original: hola
Texto en mayúsculas: HOLA

BUILD SUCCESS

Total time: 2.157 s
Finished at: 2026-09-23T15:18:43-05:00

```

CLASE RANDOM:

La clase Random de Java, perteneciente al paquete (java.util), se utiliza para generar números pseudoaleatorios dentro de un programa. Estos valores parecen aleatorios, aunque en realidad son producidos mediante un algoritmo que parte de un valor inicial conocido como semilla.

La clase Random incluye diferentes métodos que permiten generar valores de distintos tipos de datos, como números enteros (int), decimales (double y float), valores booleanos, entre otros. Esto permite utilizar datos aleatorios en juegos, sorteos, simulaciones y diferentes ejercicios de programación.

Existen diferencias entre los números verdaderamente aleatorios y los números pseudoaleatorios. Los números aleatorios no siguen un patrón predecible y se producen mediante fenómenos físicos o situaciones naturales. Por ejemplo, al lanzar un dado varias veces, cada resultado puede variar sin seguir un patrón determinado.

En cambio, los números pseudoaleatorios utilizados por la clase Random son generados mediante cálculos matemáticos. Aunque sus resultados parecen aleatorios, pueden reproducirse si se utiliza la misma semilla inicial.

DECLARACION DE METODO:

- Para utilizar la clase Random y sus métodos, primero es necesario importarla desde el paquete **import java.util.Random;**
- Una vez importada la clase Random, se debe crear una instancia u objeto para poder utilizar sus métodos **Random randomNumbers = new Random();**

Existen 22 métodos en la clase (java.util.Random) pero a continuación mencionare las más importantes y utilizadas:

- nextInt() → Genera un número entero aleatorio.
- nextInt(limite) → Genera un entero dentro de un límite.
- nextDouble() → Genera un decimal entre 0.0 y 1.0.
- nextFloat() → Genera un decimal de tipo float.
- nextLong() → Genera un número entero de tipo long.
- nextBoolean() → Genera true o false.
- nextBytes() → Genera valores aleatorios en un arreglo de bytes.
- nextGaussian() → Genera números siguiendo una distribución normal.
- setSeed() → Establece la semilla del generador aleatorio.

EJEMPLOS:

```
1 import java.util.Random;
2
3 public class ejemplos {
4
5     public static void main(String[] args) {
6
7         Random aleatorio = new Random();
8
9         int numero = aleatorio.nextInt(10) + 1;
10
11         System.out.println("Número aleatorio: " + numero);
12     }
13 }
```

Output - Run (ejemplos) x

--- exec:3.6.3:exec (default-cli) @ ejemplo_Scanner ---
Número aleatorio: 3
BUILD SUCCESS
Total time: 2.963 s
Finished at: 2026-09-24T12:41:57-05:00

ESTE CODIGO PUEDE GENERAR UN
NUMERO ALEATORIO ENTRE EL 1 Y
10.

```
1 import java.util.Random;
2
3 public class ejemplos {
4
5     public static void main(String[] args) {
6
7         Random aleatorio = new Random();
8
9         boolean moneda = aleatorio.nextBoolean();
10
11         if (moneda) {
12             System.out.println("Resultado: Cara");
13         } else {
14             System.out.println("Resultado: Sello");
15         }
16     }
17 }
```

Output - Run (ejemplos) x

--- exec:3.6.3:exec (default-cli) @ ejemplo_Scanner ---
Resultado: Cara
BUILD SUCCESS
Total time: 2.799 s
Finished at: 2026-09-24T12:44:45-05:00

ESTE CODIGO USA nextBoolean()
PARA OBTENER TRUE O FALSE,
REPRESENTANDO CARA O SELLO

```
1 import java.util.Random;
2
3 public class ejemplos {
4
5     public static void main(String[] args) {
6
7         Random aleatorio = new Random();
8
9         long numero = aleatorio.nextLong();
10
11         System.out.println("Número long aleatorio: " + numero);
12     }
13 }
```

Output - Run (ejemplos) x

--- exec:3.6.3:exec (default-cli) @ ejemplo_Scanner ---
Número long aleatorio: -1563141025123056850
BUILD SUCCESS
Total time: 1.806 s
Finished at: 2026-09-24T12:48:15-05:00

EL CODIGO nextLong() GENERA
UN NÚMERO ENTERO ALEATORIO
DE TAMAÑO GRANDE

setSeed() FIJA UN VALOR INICIAL
PARA GENERAR NÚMEROS
ALEATORIOS. SI SE USA EL MISMO
VALOR, SE PUEDE REPETIR LA
MISMA SECUENCIA DE NÚMEROS.

```
1 import java.util.Random;
2
3 public class ejemplos {
4
5     public static void main(String[] args) {
6
7         Random aleatorio = new Random();
8
9         aleatorio.setSeed(100);
10
11         System.out.println(aleatorio.nextInt(100));
12         System.out.println(aleatorio.nextInt(100));
13     }
14 }
```

Output - Run (ejemplos) x

15
50
BUILD SUCCESS
Total time: 1.742 s
Finished at: 2026-09-24T12:50:52-05:00